PROJET · ÉLECTRONIQUE NUMÉRIQUE

Concevoir un mini-processeur 16 bits en VHDL

De la feuille de papier à la carte FPGA Spartan-6 : architecture, machine d'état, simulation et affichage en temps réel.

8	16	12	3
instructions	bits de données	bits d'adresse	cycles / instruction



Écran LCD 16×2 piloté par le FPGA : résultat final 0027h = 39

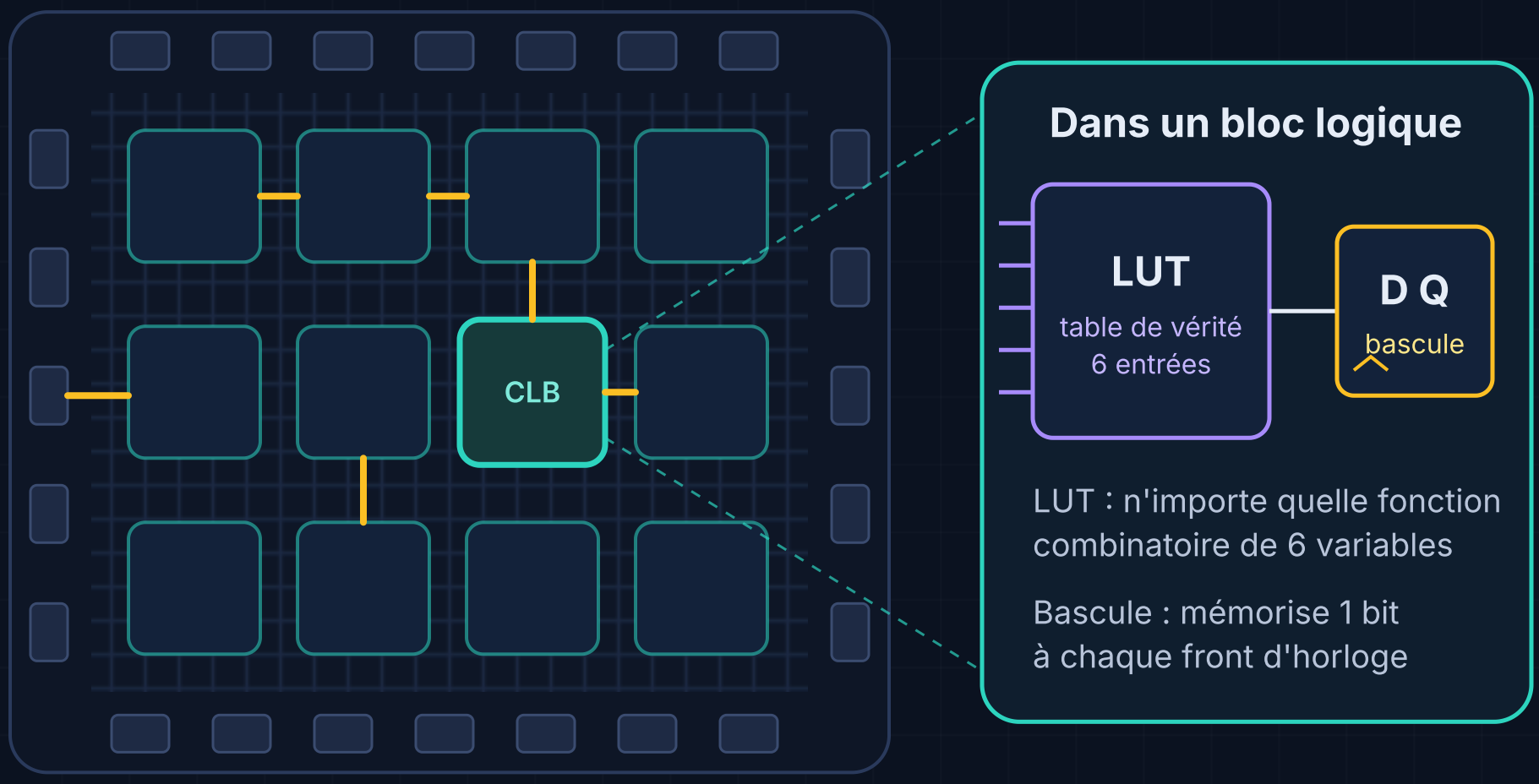
VHDL · FPGA Spartan-6 · Machine d'état · ModelSim · GHDL · Xilinx ISE

Nourhen Bahrouni

Glissez →

C'est quoi une carte FPGA ?

FPGA = *Field Programmable Gate Array*. C'est une puce remplie de blocs logiques **vides** dont on décide le câblage avec un fichier de configuration (le *bitstream*). On ne lui donne pas un programme à exécuter : on **fabrique un circuit** à l'intérieur.



blocs logiques (CLB) · broches d'entrée/sortie · liaisons configurées

Microcontrôleur

Le matériel est fixe. Il exécute un logiciel, une instruction après l'autre.

FPGA

C'est le matériel qu'on décrit. Tout fonctionne en parallèle. Ici : **un processeur entier construit dans la puce.**

Les niveaux d'abstraction

Un même processeur se décrit à plusieurs niveaux, du plus abstrait au plus physique. Le projet les traverse tous.

NIVEAU 5
Programme
Code assembleur : `LDA` `ADD` `JNE` `STP`. Ici le programme calcule 2X+Y.

NIVEAU 4
Jeu d'instructions
8 instructions. Format : opcode 4 bits + adresse 12 bits. Machine « à une adresse ».

NIVEAU 3
Architecture
Chemin de données (PC, IR, ACC, UAL, MUX, mémoire) + unité de contrôle (machine d'état).

NIVEAU 2
RTL (VHDL)
Registres et logique en VHDL **comportemental** : `process(clk)`, `case alufs is` …

NIVEAU 1
Portes logiques
Machine d'état en **flot de données** : `d1 <= (q2 and q0) or …`

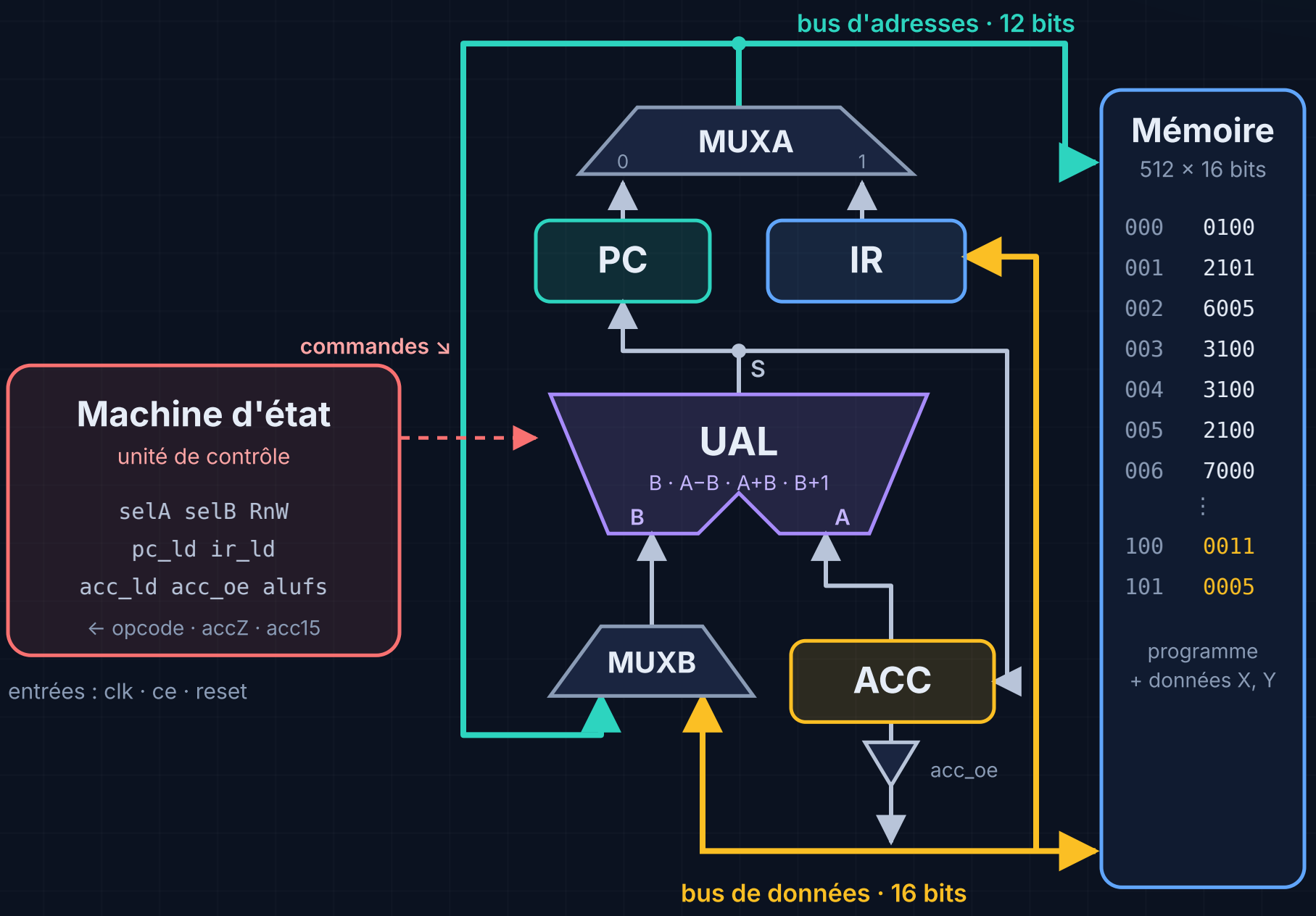
NIVEAU 0
Physique
LUT, bascules et routage du Spartan-6, configurés par le bitstream.

↑ plus abstrait

plus proche du silicium ↓

Le chemin de données

Les données circulent entre registres, UAL et mémoire. Les multiplexeurs aiguillent ; la machine d'état commande tout.



MUXA

Choisit l'adresse : **PC** pour lire l'instruction, **IR** pour l'opérande.

MUXB

Choisit l'entrée B de l'UAL : l'**adresse** (PC+1, saut) ou la **donnée** lue.

Porte 3 états

Met l'ACC sur le bus de données seulement pendant `STO`.

8 instructions, 1 format



Opérande implicite : l'accumulateur. Exemple : `ADD @101h` fait ACC ← ACC + mem[101h].

Instruction	Code	Action
`LDA adr`	`0000`	ACC ← mem[adr]
`STO adr`	`0001`	mem[adr] ← ACC
`ADD adr`	`0010`	ACC ← ACC + mem[adr]
`SUB adr`	`0011`	ACC ← ACC − mem[adr]
`JMP adr`	`0100`	PC ← adr
`JGE adr`	`0101`	si ACC ≥ 0 : PC ← adr
`JNE adr`	`0110`	si ACC ≠ 0 : PC ← adr
`STP`	`0111`	arrêt du processeur

Programme de test

```
0  LDA @100h   ; A = X
1  ADD @101h   ; A = X+Y
2  JNE 005h
3  SUB @100h
4  SUB @100h
5  ADD @100h
6  STP
```

Que calcule-t-il ?

Si X+Y ≠ 0 : **A = 2X + Y**

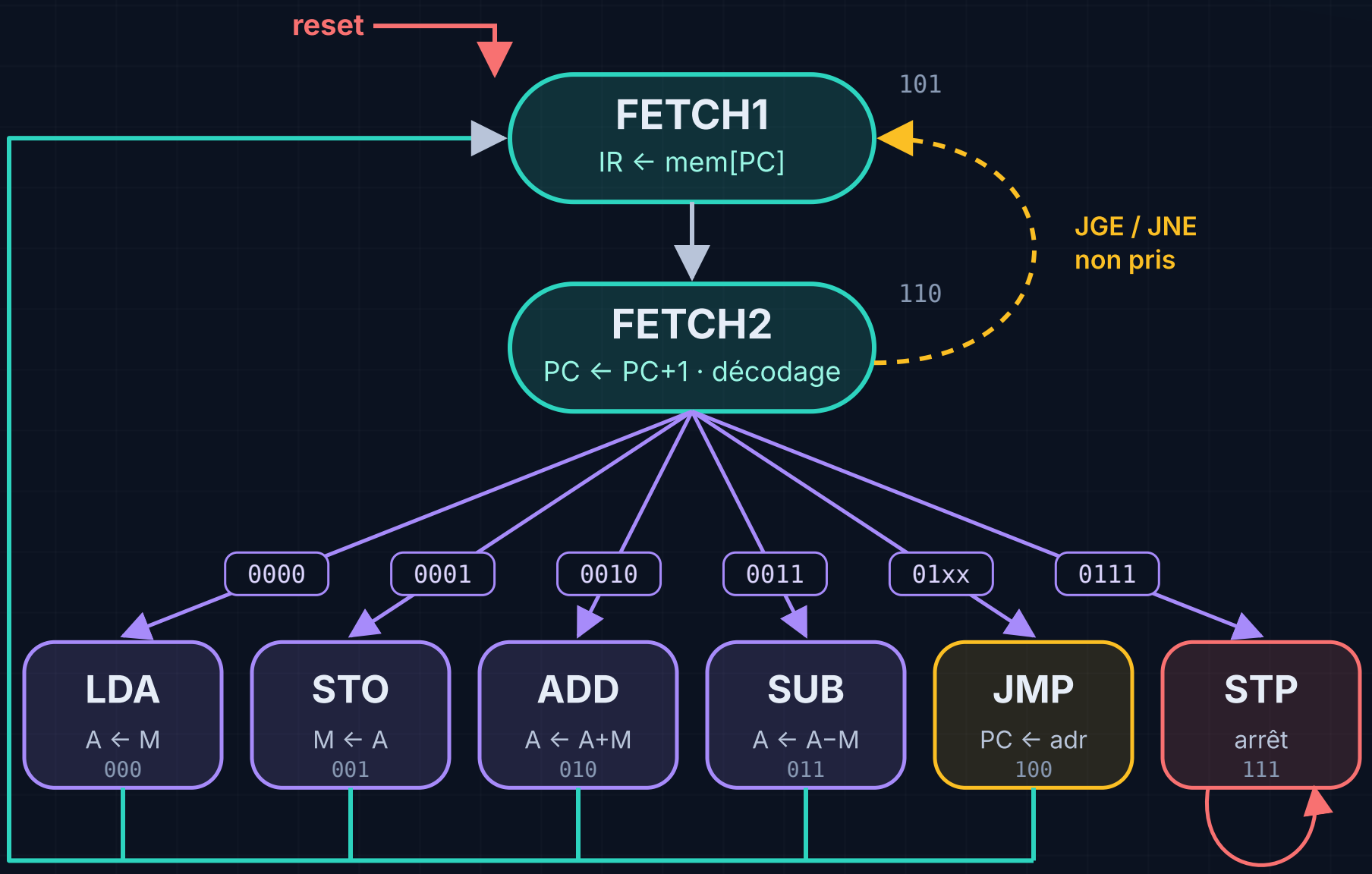
Sinon : **A = Y**

Avec X = 17, Y = 5 :

`A = 39 = 0027h`

La machine d'état

8 états codés sur 3 bascules D (q2 q1 q0). Chaque instruction suit le même cycle : **chercher**, **décoder**, **exécuter**.



01xx = JMP, JGE (si A ≥ 0) ou JNE (si A ≠ 0). Le saut passe par l'état JMP.

Codée en équations logiques (VHDL flot de données)

```
d1    <= (q2 and q0) or (q2 and q1 and n1);   -- état suivant
pc_ld <= q2 and (not q0);                     -- FETCH2, JMP
acc_ld<= (not q2 and not q0) or (not q2 and q1);
```

Ralentir le processeur pour le voir

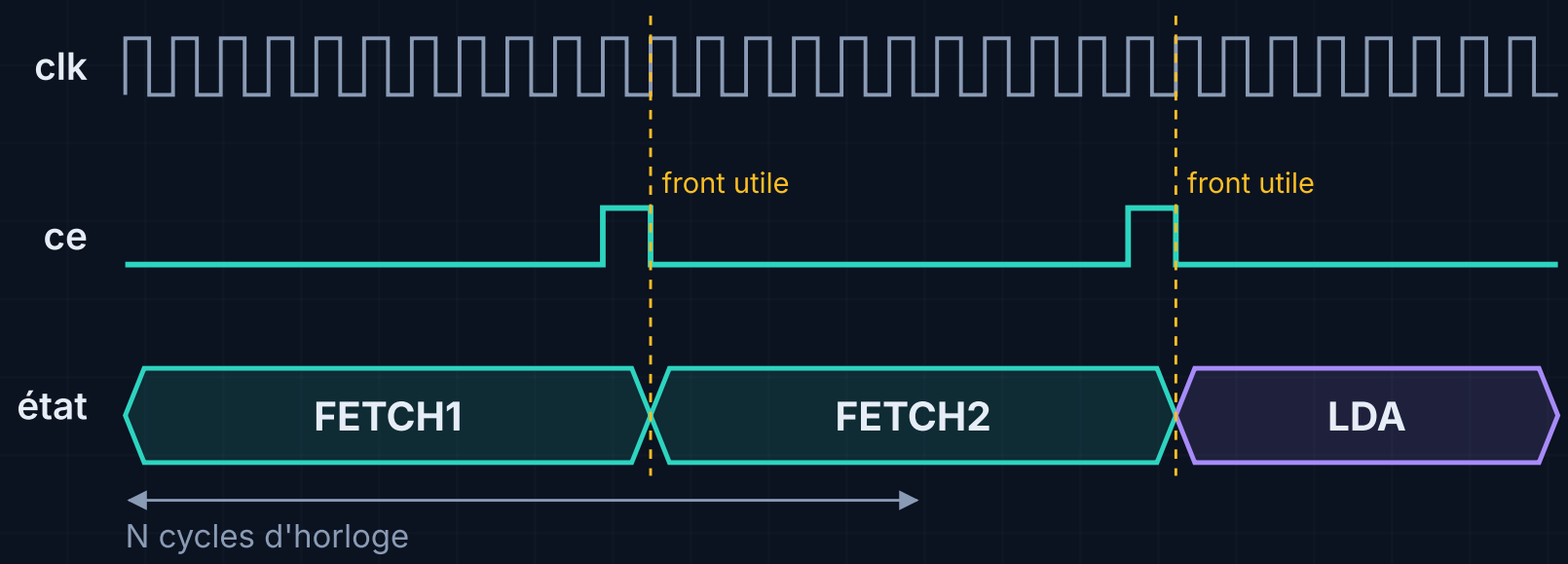
La carte tourne à **50 MHz** : 50 millions de cycles par seconde, trop vite pour l'œil. Il faut **1 cycle par seconde**.

✕ Diviseur d'horloge

Un compteur fabrique `clk_lent`, utilisée comme horloge. Ça simule bien, mais sur FPGA c'est fragile : décalages d'horloge, horloge hors du réseau global.

✓ Clock enable

Une seule horloge pour tout. Un compteur produit une impulsion `ce`. Les registres ne bougent que si `ce = '1'`.



$$N = f_{clk} \times T = 50 \cdot 10^6 \times 1\ s$$

Un compteur de 0 à N−1 donne **1 impulsion par seconde**. En mode **pas à pas**, `ce` vient d'un bouton (avec anti-rebond).

Le motif VHDL, dans chaque registre

```
if rising_edge(clk) then                    -- 50 MHz
  if ce = '1' and acc_ld = '1' then
    reg_acc <= acc_in;                      -- 1 fois / s
```

Voir PC, A et B en direct

Le processeur avance à 1 Hz, mais l'affichage tourne à pleine vitesse : il suit les registres en temps réel.

Écran LCD 16×2 (contrôleur HD44780)

Initialisation, puis réécriture des 32 caractères en boucle (~2 ms)

```
PC=005  FETCH1
A=0016  B=0005
```

PC : adresse **état** de la FSM **A** : accumulateur **B** : entrée B de l'UAL

Variante : 4 afficheurs 7 segments

Un seul digit est allumé à la fois (encadré), 1 ms chacun. L'œil voit les 4 en même temps (persistance rétinienne). Des interrupteurs choisissent A, B, PC ou IR.

A =

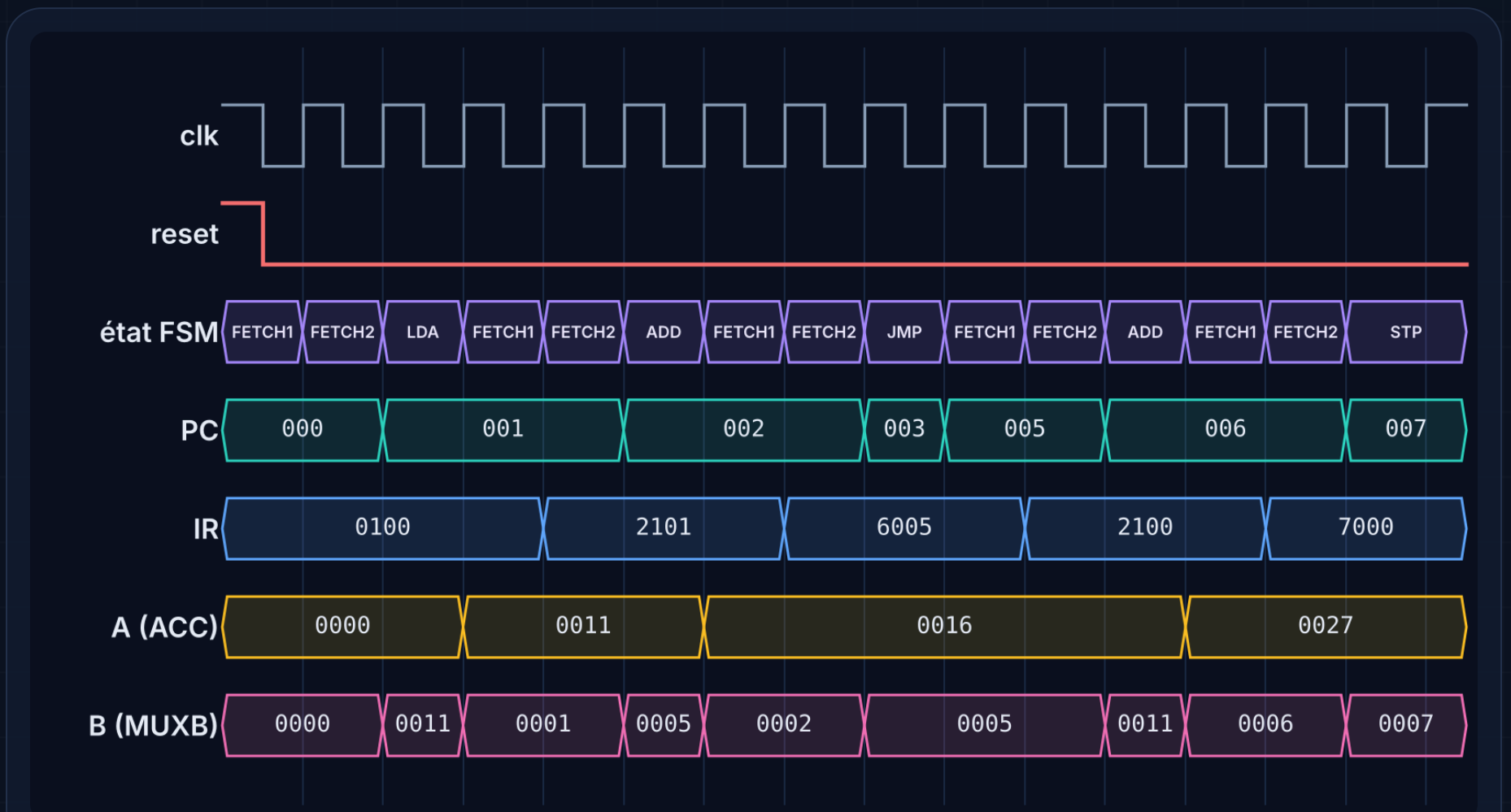
sw = 00



1 Hz	1 kHz	~2 ms
cycle processeur	balayage 7 segments	rafraîchissement LCD

Prouver que ça marche

Avant la carte, chaque bloc est vérifié en simulation par des testbenches automatiques.



- **Processeur seul :** les 2 branches de JNE sont testées (X+Y ≠ 0 donne 39, X+Y = 0 donne −5).
- **Afficheurs 7 segments :** le testbench relit les segments et reconstitue la valeur affichée.
- **Écran LCD :** un modèle du HD44780 reconstitue les 2 lignes écrites par le FPGA.
- **Intégration continue :** à chaque *push* sur GitHub, les 3 testbenches tournent automatiquement (GHDL).

Du code VHDL à la carte

Six étapes, de la description au circuit configuré.

1. **Description VHDL** — Éditeur
   Comportemental + flot de données, découpé en entités
2. **Simulation** — ModelSim · GHDL
   Testbenches, chronogrammes, assertions
3. **Synthèse** — XST (ISE)
   Le VHDL devient des LUT, bascules et RAM
4. **Placement & routage** — PAR (ISE)
   Les blocs sont placés dans la puce et reliés, broches fixées par le fichier UCF
5. **Bitstream** — BitGen
   Le fichier .bit qui configure le FPGA
6. **Test sur carte** — iMPACT · Adept
   Programmation via USB, observation sur LCD / 7 segments

Ce que ce projet m'a appris

- **Penser matériel, pas logiciel.** En VHDL, chaque ligne devient un circuit, et tout tourne en parallèle.
- **« Ça marche en simulation » ne suffit pas.** Horloges, polarités, contraintes de broches : la carte a ses propres règles.
- **Une seule horloge + clock enable** : la bonne pratique pour ralentir un circuit sur FPGA.
- **Vérifier automatiquement.** Des testbenches qui comparent au résultat attendu valent mieux qu'une lecture à l'œil des signaux.
- **Relier les niveaux d'abstraction** : du programme assembleur jusqu'aux LUT du FPGA.

Code source complet sur GitHub

VHDL commenté, 3 testbenches, contraintes de la carte et documentation.
Lien en commentaire.

Vos retours m'intéressent : comment ralentissez-vous vos designs pour les observer sur carte ?